Quick Reference Handbook

DEVOPS ENGINEERING MASTERY — Volume 06, Companion Volume. First Edition, 9 November 2026. Bengali-English glossary (180+ terms), Docker CLI, Dockerfile and Compose YAML references, networking and storage references, checklists, troubleshooting decision trees, the competency tracking sheet and official references — keep this beside your Docker host. Docker Engine, Docker Compose v2, Node.js 20.

অংশ G — শব্দকোষ (১)

বাংলা-ইংরেজি শব্দকোষ — পদ ১–৬০

প্রতিটি পদের ইংরেজি নামের পাশে বাংলা ব্যাখ্যা দেওয়া হয়েছে। প্রযুক্তিগত নাম ইংরেজিতেই রাখা হয়েছে।

#	English Term	বাংলা ব্যাখ্যা
১	Container	Kernel-স্তরের isolation-সহ চলমান প্রসেস-সমষ্টি; নিজের kernel নেই, host kernel ভাগ করে।
২	Image	Read-only লেয়ারের সংগ্রহ ও metadata; যা থেকে কনটেইনার তৈরি হয়।
৩	Docker Engine	Docker-এর মূল runtime — daemon, API ও CLI-র সমন্বয়।
৪	Docker Daemon (dockerd)	পেছনে চলা সার্ভিস; কনটেইনার, ইমেজ, নেটওয়ার্ক ও ভলিউম পরিচালনা করে।
৫	Docker Client	CLI টুল (`docker`); daemon-এর সঙ্গে API-তে কথা বলে।
৬	containerd	কনটেইনার lifecycle পরিচালনা করা নিম্ন-স্তরের runtime।
৭	runc	OCI অনুযায়ী আসল কনটেইনার প্রসেস শুরু করা runtime।
৮	OCI	Open Container Initiative — ইমেজ ও runtime-এর উন্মুক্ত মান নির্ধারণকারী সংস্থা।
৯	Namespace	Linux kernel-এর isolation ব্যবস্থা; প্রসেস, নেটওয়ার্ক, ফাইল আলাদা করে।
১০	cgroups	Control groups — CPU, মেমরি, I/O-র ব্যবহার সীমিত ও হিসাব করার ব্যবস্থা।
১১	PID Namespace	প্রসেস ID-র আলাদা দৃষ্টিভঙ্গি দেওয়া namespace।
১২	Mount Namespace	Filesystem দৃষ্টিভঙ্গি আলাদা করা namespace।
১৩	Network Namespace	ইন্টারফেস, IP ও রাউটিং আলাদা করা namespace।
১৪	User Namespace	ব্যবহারকারী ID আলাদা করা namespace; root ম্যাপিংয়ে নিরাপত্তা দেয়।
১৫	UTS Namespace	হোস্টনেম ও ডোমেইন নাম আলাদা করা namespace।
১৬	IPC Namespace	Shared memory ও semaphore আলাদা করা namespace।
১৭	Virtual Machine (VM)	Hypervisor-এর উপর চলা পূর্ণ ভার্চুয়াল কম্পিউটার; নিজের kernel থাকে।

১৮	Hypervisor	VM তৈরি ও পরিচালনা করা সফটওয়্যার স্তর।
১৯	Layer	Image-এর একটি read-only স্তর; প্রতিটি Dockerfile নির্দেশনা থেকে তৈরি হয়।
২০	Base Image	Dockerfile-এ `FROM` দিয়ে নেওয়া শুরুর image।
২১	Dockerfile	Image তৈরির ধাপ লেখা টেক্সট ফাইল।
২২	Build Context	Build সময়ে daemon-এ পাঠানো ফাইলের সংগ্রহ।
২৩	Build Cache	আগের build-এর লেয়ার পুনঃব্যবহার করে build দ্রুত করার ব্যবস্থা।
২৪	Multi-stage Build	একাধিক `FROM` stage ব্যবহার করে ছোট চূড়ান্ত image তৈরি করা।
২৫	Registry	Docker image সংরক্ষণ ও বিতরণের সার্ভিস।
২৬	Tag	Image-এর সংস্করণ চিহ্নিতকরণ নাম, যেমন `1.4.0`।
২৭	Digest	Image content-এর অপরিবর্তনীয় hash (`sha256:...`)।
২৮	Repository	একই image-এর বিভিন্ন tag-এর সংগ্রহ।
২৯	Volume	কনটেইনারের বাইরে স্থায়ী ডেটা রাখার Docker-পরিচালিত সংরক্ষণ।
৩০	Bind Mount	হোস্টের নির্দিষ্ট পথ কনটেইনারে মাউন্ট করা।
৩১	tmpfs Mount	মেমরিতে থাকা অস্থায়ী mount; রিস্টার্টে মুছে যায়।
৩২	Bridge Network	এক হোস্টের কনটেইনারগুলোকে যুক্ত করা ভার্চুয়াল নেটওয়ার্ক।
৩৩	Host Network	কনটেইনার হোস্টের নেটওয়ার্ক namespace ভাগ করে।
৩৪	Overlay Network	একাধিক হোস্টের কনটেইনারকে যুক্ত করা virtual network।
৩৫	DNS	নামকে IP ঠিকানায় রূপান্তর করা ব্যবস্থা।
৩৬	Service Discovery	চলমান সার্ভিসগুলো কীভাবে একে অপরকে খুঁজে পায়, সেই ব্যবস্থা।
৩৭	Port Mapping	হোস্টের পোর্টকে কনটেইনারের পোর্টের সঙ্গে যুক্ত করা।
৩৮	Docker Compose	একাধিক কনটেইনার YAML ফাইলে ঘোষণা করে একসাথে চালানোর টুল।
৩৯	Service (Compose)	Compose ফাইলে ঘোষিত একটি কনটেইনার-সংজ্ঞা।
৪০	Healthcheck	কনটেইনার সত্যিই সাড়া দিচ্ছে কি না, তা নিয়মিত যাচাই করা।
৪১	Restart Policy	কনটেইনার ব্যর্থ হলে বা হোস্ট রিস্টার্ট হলে কী হবে, সেই নিয়ম।
৪২	Environment Variable	কনটেইনারে runtime কনফিগারেশন দেওয়ার নাম-মান জোড়া।
৪৩	Secret	পাসওয়ার্ড, token বা connection string-এর মতো সংবেদনশীল তথ্য।
৪৪	ENTRYPOINT	কনটেইনারের মূল কমান্ড নির্ধারণ করা Dockerfile নির্দেশনা।

৪৫	CMD	ENTRYPOINT-এর ডিফল্ট আর্গুমেন্ট দেওয়া Dockerfile নির্দেশনা।
৪৬	WORKDIR	পরের নির্দেশনাগুলোর কাজের ডিরেক্টরি নির্ধারণ করা।
৪৭	EXPOSE	কনটেইনার কোন পোর্টে শোনে, তা নথিভুক্ত করা metadata।
৪৮	ARG	শুধু build সময়ে ব্যবহারযোগ্য variable।
৪৯	ENV	Image ও runtime-এ থাকা environment variable।
৫০	USER	কনটেইনার কোন ব্যবহারকারী হিসেবে চলবে, তা নির্ধারণ করা।
৫১	COPY	Build context থেকে ফাইল image-এ কপি করা।
৫২	ADD	COPY-এর মতো, তবে URL ও tar extract-এর সুবিধাসহ।
৫৩	RUN	Image তৈরির সময় কমান্ড চালিয়ে লেয়ার তৈরি করা।
৫৪	FROM	Base image নির্ধারণ করা Dockerfile নির্দেশনা।
৫৫	.dockerignore	Build context থেকে ফাইল বাদ দেওয়ার তালিকা।
৫৬	BuildKit	Docker-এর আধুনিক build ইঞ্জিন; cache mount ও secret mount দেয়।
৫৭	OOMKilled	মেমরি শেষ হয়ে kernel প্রসেস মেরে ফেলেছে — exit code 137।
৫৮	Exit Code	প্রসেস থামার সময় ফেরত দেওয়া সংখ্যা; 0 মানে সফল।
৫৯	PID 1	কনটেইনারের প্রথম প্রসেস; signal গ্রহণ ও child প্রসেস সামলানোর দায়িত্ব তার।
৬০	SIGTERM / SIGKILL	থামার signal — SIGTERM নম্রভাবে থামতে বলে, SIGKILL জোর করে থামায়।

অংশ G — শব্দকোষ (২)

বাংলা-ইংরেজি শব্দকোষ — পদ ৬১–১২০

#	English Term	বাংলা ব্যাখ্যা
৬১	Capability (Linux)	Root ক্ষমতার ছোট ছোট ভাগ; যেমন `NET_BIND_SERVICE`।
৬২	Privileged Container	হোস্টের প্রায় সব ক্ষমতা পাওয়া কনটেইনার; প্রোডাকশনে নিষিদ্ধ প্রায়।
৬৩	Read-only Filesystem	কনটেইনারের root filesystem-এ লেখা বন্ধ রাখা নিরাপত্তা ব্যবস্থা।
৬৪	Non-root User	Root-এর বদলে সাধারণ ব্যবহারকারী হিসেবে কনটেইনার চালানো।
৬৫	Least Privilege	শুধু প্রয়োজনীয় ক্ষমতাই দেওয়ার নিরাপত্তা নীতি।
৬৬	Attack Surface	আক্রমণকারী যেসব জায়গা দিয়ে ঢুকতে পারে, তার মোট পরিমাণ।
৬৭	CVE	Common Vulnerabilities and Exposures — জানা দুর্বলতার তালিকার নম্বর।
৬৮	Vulnerability Scan	Image বা সিস্টেমে জানা দুর্বলতা খোঁজা।
৬৯	Trivy	Image ও filesystem-এর দুর্বলতা খোঁজার জনপ্রিয় scanner।
৭০	Grype	Anchore-এর image vulnerability scanner।
৭১	SBOM	Software Bill of Materials — image-এ কী কী উপাদান আছে, তার তালিকা।
৭২	Image Signing	Image-এর উৎস ও অখণ্ডতা যাচাই করার জন্য ডিজিটাল স্বাক্ষর।
৭৩	Immutable Artifact	একবার তৈরি হলে বদলানো যায় না এমন artifact।
৭৪	Semantic Versioning	major.minor.patch নিয়মে সংস্করণ চিহ্নিতকরণ।
৭৫	Rolling Update	ধীরে ধীরে কয়েকটি করে কনটেইনার নতুন সংস্করণে বদলানো।
৭৬	Blue-Green Deployment	দুটি পরিবেশ রেখে একবারে ট্রাফিক বদলে দেওয়া deployment।
৭৭	Canary Deployment	প্রথমে অল্প ট্রাফিকে নতুন সংস্করণ পরীক্ষা করা deployment।
৭৮	Rollback	সমস্যা হলে আগের সংস্করণে ফিরে যাওয়া।
৭৯	Logging Driver	Docker কীভাবে কনটেইনারের লগ সংগ্রহ ও পাঠাবে, সেই ব্যবস্থা।
৮০	Log Rotation	লগ ফাইল নির্দিষ্ট আকারে পৌঁছালে পুরনোটি সরিয়ে নতুন শুরু করা।
৮১	stdout / stderr	প্রসেসের স্বাভাবিক আউটপুট ও ত্রুটির আউটপুট প্রবাহ।
৮২	Observability	লগ, মেট্রিক ও trace দিয়ে সিস্টেমের ভেতরের অবস্থা বোঝা।
৮৩	Monitoring	সিস্টেমের স্বাস্থ্য ও কর্মক্ষমতা নিয়মিত পর্যবেক্ষণ করা।
৮৪	Alerting	সমস্যা ধরা পড়লে মানুষকে জানানো।

৮৫	CPU Shares	CPU প্রতিযোগিতায় কনটেইনারের আপেক্ষিক ওজন।
৮৬	CPU Quota	CPU সময়ের কঠোর ঊর্ধ্বসীমা।
৮৭	Memory Limit	কনটেইনার সর্বোচ্চ কত মেমরি ব্যবহার করতে পারবে, সেই সীমা।
৮৮	Memory Swap	RAM পূর্ণ হলে ডিস্কে swap ব্যবহারের সীমা।
৮৯	Reservation	কনটেইনারের জন্য ন্যূনতম সম্পদ সংরক্ষণ করা।
৯০	Swarm	Docker-এর নিজস্ব clustering ও orchestration ব্যবস্থা।
৯১	Orchestration	অনেক কনটেইনার একসাথে পরিচালনা, স্কেল ও পুনরুদ্ধার করা।
৯২	Kubernetes	জনপ্রিয় কনটেইনার orchestration প্ল্যাটফর্ম।
৯৩	Pod	Kubernetes-এ একসাথে চলা এক বা একাধিক কনটেইনারের একক।
৯৪	Ingress	বাইরের ট্রাফিক ক্লাস্টারের সার্ভিসে রাউট করার প্রবেশপথ।
৯৫	Reverse Proxy	বাইরের অনুরোধ গ্রহণ করে ভেতরের সার্ভিসে পাঠানো সার্ভার (যেমন Nginx)।
৯৬	Load Balancer	ট্রাফিক একাধিক সার্ভিসে ভাগ করে দেওয়া ব্যবস্থা।
৯৭	CI/CD	Continuous Integration / Continuous Delivery — স্বয়ংক্রিয় build, test ও deploy।
৯৮	Pipeline	কোড থেকে deploy পর্যন্ত স্বয়ংক্রিয় ধাপের ক্রম।
৯৯	Artifact (CI)	Build-এর ফল — image বা প্যাকেজ, যা deploy করা হয়।
১০০	Hot Reload	কোড বদলালেই অ্যাপ আবার চালু না করে ফল দেখা।
১০১	Dev Dependency	শুধু ডেভেলপমেন্ট/build-এ দরকারি প্যাকেজ; production-এ নয়।
১০২	Lockfile	Dependency-র ঠিক সংস্করণ লেখা ফাইল (`package-lock.json`)।
১০৩	Alpine Linux	খুব ছোট, নিরাপত্তা-কেন্দ্রিক Linux distribution; ছোট image-এর base।
১০৪	Distroless Image	শুধু অ্যাপ ও runtime থাকা, shell-ছাড়া ন্যূনতম image।
১০৫	Scratch Image	একদম খালি base image; statically compiled অ্যাপের জন্য।
১০৬	Union Filesystem	একাধিক লেয়ারকে একটিমাত্র filesystem দৃষ্টিভঙ্গিতে জোড়া দেওয়া।
১০৭	overlay2	Docker-এর ডিফল্ট storage driver।
১০৮	Storage Driver	Image লেয়ার ও writable layer কীভাবে ডিস্কে রাখা হবে, সেই ব্যবস্থা।
১০৯	Dangling Image	কোনো tag নেই এমন, ব্যবহৃত-না-হওয়া image।
১১০	Prune	ব্যবহৃত হচ্ছে না এমন Docker বস্তু (image, volume, network) মুছে ফেলা।
১১১	Daemon Configuration	`daemon.json` ফাইলে Docker daemon-এর সেটিং।
১১২	Unix Socket	একই মেশিনে প্রসেসগুলোর মধ্যে যোগাযোগের মাধ্যম (`docker.sock`)।

১১৩	Container Escape	কনটেইনার থেকে বেরিয়ে হোস্টে পৌঁছানো আক্রমণ।
১১৪	Seccomp	কনটেইনার কোন system call করতে পারবে, তা সীমিত করার ব্যবস্থা।
১১৫	AppArmor / SELinux	Kernel-স্তরের নিরাপত্তা নীতি; কনটেইনারের ক্ষমতা সীমিত করে।
১১৬	Rootless Docker	Root ছাড়া Docker daemon চালানো; হোস্ট root ঝুঁকি কমায়।
১১৭	Docker Hub	Docker-এর public registry।
১১৮	Private Registry	নিজের নিয়ন্ত্রণে থাকা image সংরক্ষণের সার্ভিস।
১১৯	Registry Mirror	Registry-র স্থানীয় কপি; pull দ্রুত করে।
১২০	Image Pull Policy	কখন image টানা হবে (সবসময়, না থাকলে, কখনোই নয়) — সেই নিয়ম।

অংশ G — শব্দকোষ (৩)

বাংলা-ইংরেজি শব্দকোষ — পদ ১২১–১৮০

#	English Term	বাংলা ব্যাখ্যা
১২১	Container Runtime	কনটেইনার চালানোর সফটওয়্যার (runc, containerd)।
১২২	CRI	Container Runtime Interface — Kubernetes-এর runtime-এর সঙ্গে কথা বলার মান।
১২৩	CRI-O	Kubernetes-এর জন্য হালকা container runtime।
১২৪	Podman	Daemon-ছাড়া কনটেইনার টুল; Docker-এর বিকল্প।
১২৫	Buildah	Daemon ছাড়া image তৈরি করার টুল।
১২৬	Kaniko	Daemon ছাড়া Kubernetes-এর ভেতরে image build করার টুল।
১২৭	Dockerfile Lint	Dockerfile-এর সেরা অভ্যাস যাচাই করা (যেমন Hadolint)।
১২৮	Hadolint	Dockerfile linter টুল।
১২৯	docker-compose.yml	Compose-এর সার্ভিস, নেটওয়ার্ক ও ভলিউম ঘোষণা করা YAML ফাইল।
১৩০	Profile (Compose)	Compose-এ নির্দিষ্ট সার্ভিস-সমষ্টি শুধু প্রয়োজনে চালু করার ব্যবস্থা।
১৩১	depends_on	Compose-এ সার্ভিস শুরুর ক্রম নির্ধারণ করা।
১৩২	Deploy (Compose)	Compose-এ resource limit ও replica-সংক্রান্ত সেটিং।
১৩৩	Config (Docker)	কনটেইনারে অ-সংবেদনশীল কনফিগারেশন ফাইল দেওয়ার ব্যবস্থা।
১৩৪	Init Process	PID 1 হিসেবে চলা ছোট init (যেমন `tini`); zombie প্রসেস সামলায়।
১৩৫	Zombie Process	থেমে গেছে কিন্তু parent সংগ্রহ করেনি এমন প্রসেস।
১৩৬	Graceful Shutdown	Signal পেয়ে সংযোগ বন্ধ করে ধীরে ধীরে থামা।
১৩৭	Readiness Probe	সার্ভিস ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না, তা যাচাই করা।
১৩৮	Liveness Probe	সার্ভিস বেঁচে আছে কি না, তা যাচাই করা; ব্যর্থ হলে রিস্টার্ট হয়।
১৩৯	Sidecar Container	মূল কনটেইনারের পাশে সহায়ক কাজ (লগ, proxy) করা কনটেইনার।
১৪০	Init Container	মূল কনটেইনার শুরুর আগে একবার চলা সেটআপ কনটেইনার।
১৪১	Ephemeral Container	অস্থায়ী, একবারের কাজের জন্য চালানো কনটেইনার।
১৪২	Stateless Service	নিজের ভেতরে ডেটা রাখে না এমন সার্ভিস; সহজে বদলানো যায়।
১৪৩	Stateful Service	ডেটা সংরক্ষণ করা সার্ভিস (যেমন MongoDB); volume দরকার।
১৪৪	Data Persistence	কনটেইনার মুছলেও ডেটা টিকে থাকা।

১৪৫	Backup	ডেটার নিরাপদ কপি সংরক্ষণ করা।
১৪৬	Restore	ব্যাকআপ থেকে ডেটা ফিরিয়ে আনা।
১৪৭	Disaster Recovery	বড় বিপর্যয়ের পর সার্ভিস ফিরিয়ে আনার পরিকল্পনা।
১৪৮	RTO / RPO	Recovery Time Objective / Recovery Point Objective — কতক্ষণে ও কতটুকু ডেটা হারিয়ে ফিরতে হবে।
১৪৯	Horizontal Scaling	আরও কনটেইনার যোগ করে ক্ষমতা বাড়ানো।
১৫০	Vertical Scaling	একই কনটেইনারে আরও CPU/RAM দিয়ে ক্ষমতা বাড়ানো।
১৫১	Replica	একই সার্ভিসের একাধিক চলমান কপি।
১৫২	Service Mesh	সার্ভিসগুলোর মধ্যে যোগাযোগ, নিরাপত্তা ও পর্যবেক্ষণ পরিচালনা করা স্তর।
১৫৩	TLS / SSL	নেটওয়ার্ক যোগাযোগ এনক্রিপ্ট করার প্রোটোকল।
১৫৪	MTLS	উভয় দিকে certificate যাচাই করা TLS।
১৫৫	Network Policy	কোন সার্ভিস কোন সার্ভিসের সঙ্গে কথা বলতে পারবে, সেই নিয়ম।
১৫৬	Firewall	নেটওয়ার্ক ট্রাফিক নিয়ন্ত্রণ করা ব্যবস্থা।
১৫৭	Zero Trust	কাউকে আগে থেকে বিশ্বাস না করে প্রতিটি অনুরোধ যাচাই করার নীতি।
১৫৮	Supply Chain Security	কোড থেকে image পর্যন্ত পুরো পথের নিরাপত্তা।
১৫৯	Provenance	Artifact কোথা থেকে, কীভাবে তৈরি হয়েছে, তার প্রমাণ।
১৬০	Attestation	Build বা image-এর সত্যতা নিশ্চিত করা স্বাক্ষরিত বিবৃতি।
১৬১	ECS (AWS)	Amazon Elastic Container Service — AWS-এর managed কনটেইনার সার্ভিস।
১৬২	EKS (AWS)	Amazon Elastic Kubernetes Service — AWS-এর managed Kubernetes।
১৬৩	ECR (AWS)	Amazon Elastic Container Registry — AWS-এর private registry।
১৬৪	Cloud Run (GCP)	Google-এর serverless কনটেইনার সার্ভিস।
১৬৫	Azure Container Instances	Microsoft Azure-এর serverless কনটেইনার সার্ভিস।
১৬৬	AKS (Azure)	Azure Kubernetes Service — Azure-এর managed Kubernetes।
১৬৭	GKE (GCP)	Google Kubernetes Engine — GCP-এর managed Kubernetes।
১৬৮	IBM Cloud Code Engine	IBM Cloud-এর serverless কনটেইনার সার্ভিস।
১৬৯	Serverless Container	সার্ভার পরিচালনা ছাড়াই চাহিদা অনুযায়ী কনটেইনার চালানো।
১৭০	Cold Start	বন্ধ থাকা সার্ভিস প্রথম অনুরোধে চালু হতে সময় নেওয়া।

১৭১	Autoscaling	চাহিদা অনুযায়ী স্বয়ংক্রিয়ভাবে কনটেইনার সংখ্যা বাড়ানো-কমানো।
১৭২	Resource Quota	একটি namespace বা প্রকল্পের জন্য সম্পদের ঊর্ধ্বসীমা।
১৭৩	Node (Kubernetes)	Kubernetes ক্লাস্টারের একটি worker মেশিন।
১৭৪	Cluster	একসাথে কাজ করা একাধিক মেশিনের সমষ্টি।
১৭৫	Manifest	Kubernetes বা Compose-এর ঘোষণামূলক কনফিগারেশন ফাইল।
১৭৬	Helm Chart	Kubernetes অ্যাপ প্যাকেজ করার টেমপ্লেট ব্যবস্থা।
১৭৭	GitOps	Git-কে সত্যের উৎস ধরে infrastructure ও deploy পরিচালনা করা।
১৭৮	Infrastructure as Code	কোড দিয়ে infrastructure ঘোষণা ও পরিচালনা করা।
১৭৯	Twelve-Factor App	ক্লাউড-উপযোগী অ্যাপের ১২টি নীতি; কনটেইনার-বান্ধব ডিজাইনের ভিত্তি।
১৮০	Cloud Native	কনটেইনার, microservices ও orchestration-এর উপর ভিত্তি করে অ্যাপ ডিজাইন করা।

মোট শব্দকোষ পদ: **১৮০টি** (অংশ G-১: ৬০টি, G-২: ৬০টি, G-৩: ৬০টি)।

অংশ H — কুইক রেফারেন্স: Docker CLI

Docker CLI কমান্ড রেফারেন্স

সব কমান্ড Ubuntu 24.04 LTS ও Docker Engine-এর জন্য লেখা। `technova/api` একটি কাল্পনিক image নাম।

কনটেইনার কমান্ড

কমান্ড	উদ্দেশ্য
`docker run -d --name api -p 3000:3000 technova/api:1.4.0`	পেছনে (detached) কনটেইনার চালানো, নাম ও পোর্টসহ।
`docker run -it --rm node:20-alpine sh`	অস্থায়ী interactive shell; বের হলে কনটেইনার মুছে যায়।
`docker ps` / `docker ps -a`	চলমান / সব কনটেইনারের তালিকা।
`docker logs -f api`	কনটেইনারের লগ live দেখা।
`docker logs --tail 100 --since 1h api`	শেষ ১০০ লাইন, গত ১ ঘণ্টার লগ।
`docker exec -it api sh`	চলমান কনটেইনারে shell-এ ঢোকা।
`docker exec api node -e "console.log('ok')"`	কনটেইনারে একটিমাত্র কমান্ড চালানো।
`docker stop api` / `docker start api` / `docker restart api`	কনটেইনার থামানো, চালু করা, রিস্টার্ট করা।
`docker stop -t 30 api`	৩০ সেকেন্ড graceful shutdown সময় দিয়ে থামানো।
`docker rm api` / `docker rm -f api`	থেমে যাওয়া কনটেইনার মুছে ফেলা / জোর করে মুছে ফেলা।
`docker inspect api`	কনটেইনারের সম্পূর্ণ বিবরণ (JSON)।
`docker inspect -f '{{.State.Health.Status}}' api`	শুধু health status দেখা।
`docker stats` / `docker stats api`	CPU, মেমরি, I/O-র live ব্যবহার।
`docker top api`	কনটেইনারের ভেতরের প্রসেস তালিকা।
`docker cp api:/app/logs/app.log ./app.log`	কনটেইনার থেকে ফাইল হোস্টে কপি করা।
`docker update --memory 512m --cpus 1 api`	চলমান কনটেইনারের resource limit বদলানো।
`docker pause api` / `docker unpause api`	কনটেইনার সাময়িক থামানো / আবার চালু করা।
`docker rename api api-old`	কনটেইনারের নাম বদলানো।
`docker wait api`	

	কনটেইনার থামা পর্যন্ত অপেক্ষা করা ও exit code পাওয়া।
`docker diff api`	কনটেইনারের writable layer-এ কী বদলেছে, তা দেখা।

ইমেজ কমান্ড

কমান্ড	উদ্দেশ্য
`docker build -t technova/api:1.4.0 .`	বর্তমান ডিরেক্টরি থেকে image build করা।
`docker build --no-cache -t technova/api:1.4.0 .`	Cache ছাড়া নতুন করে build করা।
`docker build --target builder -t technova/api:build .`	Multi-stage-এর নির্দিষ্ট stage পর্যন্ত build করা।
`docker image ls`	স্থানীয় image-এর তালিকা।
`docker image inspect technova/api:1.4.0`	Image-এর বিবরণ ও metadata দেখা।
`docker history technova/api:1.4.0`	প্রতিটি লেয়ারের আকার ও নির্দেশনা দেখা।
`docker tag technova/api:1.4.0 technova/api:latest`	Image-এ নতুন tag যোগ করা।
`docker pull node:20-alpine`	Registry থেকে image টানা।
`docker push technova/api:1.4.0`	Registry-তে image পাঠানো।
`docker login` / `docker logout`	Registry-তে লগইন / লগআউট।
`docker image rm technova/api:1.4.0`	Image মুছে ফেলা।
`docker image prune -a`	ব্যবহৃত-না-হওয়া সব image মুছে ফেলা।
`docker save technova/api:1.4.0 -o api.tar`	Image tar ফাইলে সংরক্ষণ করা।
`docker load -i api.tar`	tar ফাইল থেকে image লোড করা।
`docker scout quickview technova/api:1.4.0`	Image-এর সংক্ষিপ্ত নিরাপত্তা বিবরণ (যদি উপলব্ধ থাকে)।

নেটওয়ার্ক, ভলিউম, সিস্টেম ও Compose কমান্ড

কমান্ড	উদ্দেশ্য
`docker network ls` / `docker network inspect technova-net`	নেটওয়ার্ক তালিকা / বিবরণ।
`docker network create technova-net`	User-defined bridge network তৈরি করা।

`docker network connect technova-net api`	কনটেইনারকে নেটওয়ার্কে যুক্ত করা।
`docker network rm technova-net`	নেটওয়ার্ক মুছে ফেলা।
`docker volume ls` / `docker volume inspect mongo-data`	ভলিউম তালিকা / বিবরণ।
`docker volume create mongo-data`	Named volume তৈরি করা।
`docker volume rm mongo-data`	ভলিউম মুছে ফেলা (ডেটা হারাবে)।
`docker volume prune`	ব্যবহৃত-না-হওয়া ভলিউম মুছে ফেলা — সাবধানে।
`docker system df`	Docker কত ডিস্ক ব্যবহার করছে, তা দেখা।
`docker system prune`	ব্যবহৃত-না-হওয়া বস্তু পরিষ্কার করা।
`docker system info` / `docker version`	Docker-এর সংস্করণ ও কনফিগারেশন।
`docker events --since 1h`	গত ১ ঘণ্টার Docker event দেখা।
`docker compose up -d`	Compose স্ট্যাক পেছনে চালু করা।
`docker compose up -d --build`	Image build করে স্ট্যাক চালু করা।
`docker compose ps` / `docker compose logs -f api`	স্ট্যাকের অবস্থা / নির্দিষ্ট সার্ভিসের লগ।
`docker compose down`	স্ট্যাক থামিয়ে কনটেইনার ও নেটওয়ার্ক মুছে ফেলা (ভলিউম থাকে)।
`docker compose down -v`	স্ট্যাকসহ ভলিউমও মুছে ফেলা — ডেটা হারাবে, সাবধানে।
`docker compose exec api sh`	Compose সার্ভিসে shell-এ ঢোকা।
`docker compose config`	Compose ফাইলের চূড়ান্ত রূপ যাচাই করা।
`docker compose pull` / `docker compose push`	স্ট্যাকের image টানা / পাঠানো।

> **সতর্কতা:** `prune`, `down -v` ও `rm -f` স্থায়ীভাবে মুছে ফেলে। প্রোডাকশনে চালানোর আগে কী মুছবে, তা যাচাই করুন।

অংশ H — কুইক রেফারেন্স: Dockerfile

Dockerfile নির্দেশনা রেফারেন্স

নির্দেশনা	সিনট্যাক্স ও উদ্দেশ্য	উদাহরণ	সতর্কতা
`FROM`	Base image নির্ধারণ করে; প্রতি stage-এ একবার লাগে।	`FROM node:20-alpine AS builder`	Tag ছাড়া `FROM node` মানে `latest` — এড়িয়ে চলুন।
`WORKDIR`	পরের নির্দেশনার কাজের ডিরেক্টরি নির্ধারণ করে।	`WORKDIR /app`	না দিলে `/`-তে কাজ হয়; পথ-ভুল সহজ হয়।
`COPY`	Build context থেকে ফাইল image-এ কপি করে।	`COPY package*.json ./`	`COPY . .` প্রথমে রাখলে cache ভাঙে।
`ADD`	COPY-এর মতো, তবে URL ও tar extract করে।	`ADD app.tar.gz /app/`	সাধারণ কপিতে COPY ব্যবহার করুন; ADD-এর আচরণ চমক দিতে পারে।
`RUN`	কমান্ড চালিয়ে নতুন লেয়ার তৈরি করে।	`RUN npm ci --omit=dev`	apt install ও cache পরিষ্কার একই `RUN`-এ লিখুন।
`CMD`	কনটেইনারের ডিফল্ট কমান্ড/আর্গুমেন্ট।	`CMD ["node", "server.js"]`	Shell form-এ signal সমস্যা হয়; exec form ব্যবহার করুন।
`ENTRYPOINT`	কনটেইনারের মূল কমান্ড নির্ধারণ করে।	`ENTRYPOINT ["node", "server.js"]`	CMD-সহ ব্যবহার করলে আর্গুমেন্ট বদলানো যায়।
`ENV`	Environment variable সেট করে (image ও runtime-এ থাকে)।	`ENV NODE_ENV=production`	Secret কখনো ENV-এ লিখবেন না।
`ARG`	শুধু build সময়ের variable।	`ARG APP_VERSION=1.4.0`	ARG runtime-এ থাকে না; secret-এর জন্য BuildKit secret mount ব্যবহার করুন।
`EXPOSE`	কোন পোর্টে অ্যাপ শোনে, তা নথিভুক্ত করে।	`EXPOSE 3000`	এটি পোর্ট প্রকাশ করে না; `-p` লাগে।
`USER`	কোন ব্যবহারকারী হিসেবে কনটেইনার চলবে।	`USER node`	Root-এ চালানো এড়াতে শেষের দিকে USER দিন।
`VOLUME`	Mount point ঘোষণা করে।	`VOLUME ["/data/db"]`	

			VOLUME-এর ভেতরের ডেটা পরের `COPY` -এ ঢাকা পড়তে পারে।
`HEALTHCHECK`	কনটেইনারের স্বাস্থ্য যাচাইয়ের কমান্ড।	`HEALTHCHECK --interval=30s CMD node healthcheck.js`	Healthcheck টুল (যেমন curl) image-এ থাকতে হবে।
`LABEL`	Image-এ metadata যোগ করে।	`LABEL maintainer="technova.example"`	সংস্করণ তথ্য LABEL-এ রাখা ভালো অভ্যাস।
`SHELL`	RUN-এর ডিফল্ট shell বদলায়।	`SHELL ["/bin/sh", "-c"]`	Alpine-এ `/bin/sh` ব্যবহার করুন, bash নয়।
`STOPSIGNAL`	থামার সময় কোন signal পাঠানো হবে।	`STOPSIGNAL SIGTERM`	অ্যাপ অন্য signal আশা করলে এটি দিন।
`ONBUILD`	এই image base হিসেবে ব্যবহার হলে চলা নির্দেশনা।	`ONBUILD COPY . /app`	অপ্রত্যাশিত আচরণ হয়; আধুনিক ব্যবহারে এড়িয়ে চলুন।

TechNova API-র নমুনা multi-stage Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=builder /app/dist ./dist
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD node dist/healthcheck.js
CMD ["node", "dist/server.js"]
```

`.dockerignore` -এ রাখুন: `node_modules`, `.git`, `.env`, `*.log`, `dist` (build-এ তৈরি হয়)।

অংশ H — কুইক রেফারেন্স: Docker Compose

Docker Compose YAML রেফারেন্স (আধুনিক v2 সিনট্যাক্স)

আধুনিক Compose-এ `version:` key ব্যবহার করা হয় না — এটি obsolete। নিচের উদাহরণে কোনো `version:` নেই।

```
services:
  web:
    build: ./web
    image: technova/web:1.4.0
    ports:
      - "3000:3000"
    environment:
      - NEXT_PUBLIC_API_URL=http://api:4000
    depends_on:
      api:
        condition: service_healthy
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
    restart: unless-stopped
    profiles: ["full"]
  api:
    build: ./api
    image: technova/api:1.4.0
    ports:
      - "4000:4000"
    env_file:
      - ./api.env
    environment:
      - NODE_ENV=production
    depends_on:
      mongo:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "node", "dist/healthcheck.js"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
    volumes:
      - api-logs:/app/logs
    networks:
      - technova-net
    restart: unless-stopped
  mongo:
    image: mongo:7
    volumes:
      - mongo-data:/data/db
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-MONGO-PASSWORD
    healthcheck:
```

```
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - technova-net
    restart: unless-stopped
networks:
  technova-net:
    driver: bridge
volumes:
  mongo-data:
  api-logs:
secrets:
  mongo_password:
    file: ./secrets/mongo_password.txt
configs:
  nginx_conf:
    file: ./nginx.conf
```

মূল কীগুলো

কী	ব্যাখ্যা
`services:`	চালাতে হবে এমন প্রতিটি কনটেইনারের সংজ্ঞা।
`build:` / `image:`	Dockerfile থেকে build করা বা registry থেকে image টানা।
`ports:`	হোস্ট:কনটেইনার পোর্ট ম্যাপিং।
`environment:` / `env_file:`	সরাসরি variable বা ফাইল থেকে variable; env ফাইল Git-এ কমিট করবেন না।
`depends_on:` + `condition: service_healthy`	শুরুর ক্রম ও readiness-ভিত্তিক নির্ভরতা।
`healthcheck:`	test, interval, timeout, retries, start_period।
`volumes:`	Named volume বা bind mount।
`networks:`	কোন network-এ সার্ভিস যুক্ত হবে।
`deploy.resources.limits`	CPU ও মেমরি সীমা (Swarm-এ পূর্ণ কার্যকর; Compose-এ `deploy` সমর্থিত)।
`restart:`	`no`, `always`, `on-failure`, `unless-stopped`।
`profiles:`	শুধু `--profile full` দিলে চালু হওয়া সার্ভিস।
`secrets:` / `configs:`	Swarm/Compose-এ secret ও কনফিগারেশন ফাইল দেওয়া; secret কখনো image-এ নয়।
`command:` / `entrypoint:`	ডিফল্ট কমান্ড বদলানো।

অংশ H — কুইক রেফারেন্স: নেটওয়ার্কিং ও স্টোরেজ

কনটেইনার নেটওয়ার্কিং ও স্টোরেজ রেফারেন্স

নেটওয়ার্কিং

ড্রাইভার	ব্যবহার	মনে রাখার কথা
bridge (user-defined)	এক হোস্টে সার্ভিসগুলোর মধ্যে যোগাযোগ; ডিফল্ট পছন্দ।	কনটেইনার নামে DNS resolution হয়।
bridge (default)	কোনো network না দিলে ব্যবহৃত হয়।	নামে DNS resolution হয় না — এড়িয়ে চলুন।
host	কনটেইনার হোস্টের নেটওয়ার্ক ভাগ করে।	Isolation কমে; পোর্ট সংঘাত হয়।
none	কোনো নেটওয়ার্ক নেই।	একবারের বিচ্ছিন্ন কাজে।
overlay	একাধিক হোস্টের কনটেইনার এক network-এ।	Swarm/Kubernetes-এ ব্যবহৃত হয়।
macvlan	কনটেইনারকে সরাসরি physical network-এ IP দেওয়া।	বিশেষ নেটওয়ার্ক প্রয়োজনে।

DNS: একই user-defined network-এ কনটেইনার নাম resolve হয় embedded DNS (127.0.0.11) দিয়ে। বাইরের নামের জন্য হোস্টের DNS বা `--dns` ব্যবহৃত হয়। **পোর্ট:** `-p হোস্ট:কনটেইনার` ; শুধু ভেতরের যোগাযোগের জন্য `expose:` যথেষ্ট। **Ingress:** বাইরের ট্রাফিক Nginx reverse proxy দিয়ে গ্রহণ করে ভেতরের সার্ভিসে রাউট করুন।

স্টোরেজ ও ভলিউম

ধরন	কখন ব্যবহার করবেন	উদাহরণ
Named volume	ডেটাবেসের মতো স্থায়ী ডেটা; Docker পরিচালনা করে।	`-v mongo-data:/data/db`
Bind mount	Dev-এ সোর্স কোড শেয়ার; হোস্টের নির্দিষ্ট পথ।	`-v ./src:/app/src`
tmpfs mount	অস্থায়ী, সংবেদনশীল ডেটা; মেমরিতে থাকে।	`--tmpfs /app/cache`
Writable layer	অস্থায়ী, কনটেইনার-নির্দিষ্ট লেখা।	কনটেইনার মুছলে মুছে যায়।

ব্যাকআপ: `docker run --rm -v mongo-data:/data -v $(pwd):/backup alpine tar czf /backup/mongo-backup.tar.gz /data` — volume মাউন্ট করে tar ব্যাকআপ নিন; restore-এর আগে পরীক্ষা করুন। **সতর্কতা:** `down -v` ও `volume prune` স্থায়ীভাবে ডেটা মুছে ফেলে।

অংশ I — চেকলিস্ট

চেকলিস্ট — Image, নিরাপত্তা, প্রোডাকশন, ব্যাকআপ

Image Optimization চেকলিস্ট

- Base image ছোট ও নির্দিষ্ট tag-সহ (যেমন `node:20-alpine`), `latest` নয়।
- Multi-stage build: build টুল চূড়ান্ত image-এ নেই।
- `package*.json` আগে কপি, তারপর `npm ci`, তারপর সোর্স — cache-বান্ধব ক্রম।
- `.dockerignore`-এ `node_modules`, `.git`, `.env`, লগ আছে।
- Production-এ `npm ci --omit=dev`; cache পরিষ্কার করা হয়েছে।
- `docker history` দিয়ে বড় লেয়ার যাচাই করা হয়েছে।
- একটি `RUN`-এ apt install ও cache পরিষ্কার একসাথে।

Container Security চেকলিস্ট

- Non-root user (`USER node`) ব্যবহার করা হয়েছে।
- Root filesystem read-only যেখানে সম্ভব; লেখার জায়গায় tmpfs/volume।
- `--cap-drop=ALL` দিয়ে শুধু প্রয়োজনীয় capability যোগ করা হয়েছে।
- `--privileged` ও `docker.sock` মাউন্ট নেই।
- কোনো secret image, Dockerfile ENV বা Git-এ নেই।
- Trivy/Grype scan CI-তে চলে; critical/high পেলে build fail হয়।
- Base image নিয়মিত আপডেট করা হয়।
- Image digest/tag রেকর্ড করা হয়।

Production Deployment চেকলিস্ট

- Immutable tag/digest ব্যবহার করা হয়েছে; deploy রেকর্ড আছে।
- Healthcheck ও restart policy (`unless-stopped`) সেট করা হয়েছে।
- Resource limit (CPU/মেমরি) প্রতিটি সার্ভিসে দেওয়া হয়েছে।
- Log rotation (`max-size` / `max-file`) সেট করা হয়েছে।
- Monitoring ও alerting চালু আছে; কেউ না জানলে থেমে যাওয়া ধরা পড়ে।
- Rollback পরিকল্পনা লেখা ও পরীক্ষা করা হয়েছে।
- ব্যাকআপ নেওয়া হয়েছে ও restore পরীক্ষা করা হয়েছে।
- Staging-এ একই image পরীক্ষা করা হয়েছে।

Backup ও Restore চেকলিস্ট

- MongoDB volume-এর নিয়মিত ব্যাকআপ (tar বা `mongodump`) নেওয়া হয়।
- ব্যাকআপ হোস্টের বাইরে সংরক্ষিত হয়।

- Restore ধাপ লেখা আছে ও নিয়মিত পরীক্ষা করা হয়।
- ব্যাকআপের আগে কনটেইনার থামানো বা consistent snapshot নেওয়া হয় কি না, তা জানা আছে।
- `down -v` প্রোডাকশনে নিষিদ্ধ — দলে সবাই জানে।

অংশ J — ট্রাবলশুটিং ডিসিশন ট্রি

কনটেইনার ট্রাবলশুটিং ডিসিশন ট্রি

১. কনটেইনার শুরু হচ্ছে না

```
docker ps -a → Exited (?) দেখুন
  ├─ Exit 1 → docker logs → অ্যাপের ত্রুটি (module নেই? env ভুল?)
  │     └─ Dockerfile / env / COPY যাচাই করুন
  ├─ Exit 137 → OOMKilled? → docker inspect → memory limit বাড়ান / leak খুঁজুন
  ├─ Exit 125 → docker run-এর ভুল (পোর্ট, নাম, image) → কমান্ড যাচাই করুন
  └─ Restarting → healthcheck / crash loop → logs + inspect দেখুন
```

২. Image build ব্যর্থ হচ্ছে

```
কোন ধাপে ব্যর্থ হলো?
  ├─ COPY ব্যর্থ → ফাইল আছে কি? .dockerignore-এ বাদ পড়েছে কি?
  ├─ npm ci ব্যর্থ → নেটওয়ার্ক? lockfile? → npm cache / registry যাচাই
  ├─ apt ব্যর্থ → update ও install এক RUN-এ আছে কি?
  └─ ধীর build → cache ভাঙছে কি? → COPY ক্রম ও .dockerignore যাচাই
```

৩. নেটওয়ার্কিং / DNS ব্যর্থতা

```
কনটেইনার থেকে ping/curl করুন
  ├─ নাম resolve হয় না → একই network-এ আছে কি? → user-defined network ব্যবহার করুন
  ├─ IP-তে হয়, নামে হয় না → embedded DNS সমস্যা → network inspect করুন
  ├─ বাইরে যেতে পারে না → DNS (--dns) / egress / proxy যাচাই
  └─ পোর্টে সাড়া নেই → -p ম্যাপিং ঠিক আছে কি? অ্যাপ 0.0.0.0-তে শুনছে কি?
```

৪. ডেটা হারিয়ে গেছে

```
কনটেইনার মুছেছেন?
  ├─ down -v / volume prune চালিয়েছেন? → ব্যাকআপ থেকে restore করুন
  ├─ volume ছিল না → writable layer-এর ডেটা গেছে → ভবিষ্যতে volume ব্যবহার করুন
  └─ volume আছে কিন্তু খালি → ভুল volume mount হয়েছে কি? → inspect দিয়ে পথ যাচাই
```

৫. Performance / OOM

```
docker stats → কোন সম্পদ বেশি?
  ├─ মেমরি 100% → limit কত? → inspect-এ OOMKilled? → leak / cache যাচাই
  ├─ CPU 100% → কোন প্রসেস? → docker top → limit দিন
  └─ ডিস্ক পূর্ণ → docker system df → লগ / image পরিষ্কার করুন
```

৬. Registry push/pull ব্যর্থতা

```
docker push/pull-এর ত্রুটি পড়ুন
  ├─ denied → docker login করেছেন কি? token-এর মেয়াদ আছে কি?
  ├─ not found → repository/tag-এর নাম ঠিক আছে কি?
  ├─ timeout → নেটওয়ার্ক / registry mirror যাচাই
  └─ manifest unknown → tag/digest ঠিক আছে কি?
```

অংশ K — দক্ষতা ট্র্যাকিং

ব্যক্তিগত দক্ষতা ট্র্যাকিং শিট (Competency Tracking Sheet)

প্রতিটি দক্ষতায় নিজের স্তর চিহ্নিত করুন: **শুরু হয়নি / শিখছি / অনুশীলন করেছি / আত্মবিশ্বাসী**। প্রতি সপ্তাহে আপডেট করুন।

দক্ষতা	শুরু হয়নি	শিখছি	অনুশীলন করেছি	আত্মবিশ্বাসী	প্রমাণ / নোট
VM বনাম কনটেইনার ব্যাখ্যা করা	☐	☐	☐	☐	
Namespace ও cgroups ব্যাখ্যা করা	☐	☐	☐	☐	
Docker CLI (run, ps, logs, exec, inspect)	☐	☐	☐	☐	
Dockerfile লেখা (cache-বান্ধব ক্রম)	☐	☐	☐	☐	
Multi-stage build	☐	☐	☐	☐	
Image build, tag, push/pull	☐	☐	☐	☐	
Bridge network ও DNS troubleshooting	☐	☐	☐	☐	
Volume ও ডেটা স্থায়িত্ব	☐	☐	☐	☐	
Docker Compose স্ট্যাক লেখা	☐	☐	☐	☐	
MERN স্ট্যাক কনটেইনারাইজেশন	☐	☐	☐	☐	
Secrets নিরাপদে পরিচালনা	☐	☐	☐	☐	
Non-root, read-only, capability drop	☐	☐	☐	☐	
Image scanning (Trivy)	☐	☐	☐	☐	
Resource limit ও docker stats	☐	☐	☐	☐	
Logging ও log rotation	☐	☐	☐	☐	
Healthcheck ও restart policy	☐	☐	☐	☐	
Deployment ও rollback	☐	☐	☐	☐	
Backup ও restore পরীক্ষা	☐	☐	☐	☐	
Incident response (triage → post-incident)	☐	☐	☐	☐	
ক্যাপস্টোন: TechNova MERN স্ট্যাক সম্পূর্ণ	☐	☐	☐	☐	

স্ব-মূল্যায়ন প্রশ্ন: (১) কোন দক্ষতায় আমি এখনো আটকে আছি? (২) কোন প্রমাণ (লগ, আউটপুট, image) দেখাতে পারব? (৩) আগামী সপ্তাহে কোন একটি দক্ষতা “আত্মবিশ্বাসী”-তে নেব?

অংশ L — অফিসিয়াল রেফারেন্স

অফিসিয়াল রেফারেন্স

নিচের লিঙ্কগুলো অফিসিয়াল ডকুমেন্টেশনের — এগুলোই কর্তৃপক্ষ (authority)। সংস্করণ, দাম বা সীমা পরিবর্তন হতে পারে; অফিসিয়াল ডকুমেন্টেশনই চূড়ান্ত।

Docker ও কনটেইনার

- Docker Documentation — https://docs.docker.com/
- Docker Engine — https://docs.docker.com/engine/
- Docker Compose Documentation — https://docs.docker.com/compose/
- Dockerfile Reference — https://docs.docker.com/reference/dockerfile/
- Docker CLI Reference — https://docs.docker.com/reference/cli/docker/
- Docker Hub — https://hub.docker.com/
- containerd Documentation — https://containerd.io/docs/
- OCI Specifications — https://opencontainers.org/ (Image Spec: https://github.com/opencontainers/image-spec, Runtime Spec: https://github.com/opencontainers/runtime-spec)

Linux kernel ও man-pages

- Linux man-pages — https://man7.org/linux/man-pages/
- Linux Kernel Documentation — https://www.kernel.org/doc/html/latest/
- Namespaces man-page — https://man7.org/linux/man-pages/man7/namespaces.7.html
- cgroups man-page — https://man7.org/linux/man-pages/man7/cgroups.7.html

অ্যাপ স্ট্যাক

- Node.js Documentation — https://nodejs.org/en/docs
- MongoDB Documentation — https://www.mongodb.com/docs/
- PostgreSQL Documentation — https://www.postgresql.org/docs/
- Next.js Documentation — https://nextjs.org/docs
- Express Documentation — https://expressjs.com/

ক্লাউড কনটেইনার সার্ভিস

- AWS Containers Documentation — https://docs.aws.amazon.com/ecs/ ও https://docs.aws.amazon.com/eks/ (ECR: https://docs.aws.amazon.com/ecr/)
- Microsoft Azure Container Instances — https://learn.microsoft.com/en-us/azure/container-instances/ (AKS: https://learn.microsoft.com/en-us/azure/aks/)

- Google Cloud Run — https://cloud.google.com/run/docs (GKE: https://cloud.google.com/kubernetes-engine/docs)
- IBM Cloud Code Engine — https://cloud.ibm.com/docs/codeengine

নিরাপত্তা সরঞ্জাম

- Trivy Documentation — https://aquasecurity.github.io/trivy/
- Hadolint — https://github.com/hadolint/hadolint

© 2026 DevOps Engineering Mastery series — Volume 06: Docker & Container Engineering. এই বইয়ের সব উদাহরণ কাল্পনিক (TechNova, technova.example, 203.0.113.x, CHANGE-ME-...)। কোনো আসল secret, token বা পাসওয়ার্ড এই বইয়ে নেই।